SUMMARY: INTRODUCTION TO LLAMAINDEX

Objectives

By the end of this session students will be able to:

1. **Explain what LlamaIndex is** – a framework that manages how documents are chunked, indexed, and retrieved, going beyond raw vector search.
2. **Differentiate LangChain vs LlamaIndex** – LangChain as the "workflow orchestrator," LlamaIndex as the "knowledge manager."
3. **Describe a Node** – the smallest unit of knowledge (chunk + metadata) used to build indexes in LlamaIndex.
4. **List the four Index types** – VectorStoreIndex, SummaryIndex, KeywordTableIndex, TreeIndex – and what each is best for.
5. **Understand the Index Layer** – how documents are stored across different backends (Pinecone, FAISS, Qdrant, Postgres, in-memory).
6. **Explain the Embeddings + LLM Layer** – embeddings as translators into vectors, LLMs as storytellers that generate the final answer.
7. **Outline the Query Layer** – retrieval engines, query fusion, sub-questioning, routing, and post-processing (rerank, dedup, compression).
8. **Show how prompts are templated** – using grounded context, fallback rules, and inline citations to keep answers factual.

Introduction to LlamaIndex

Last time, we built **RAG pipelines** manually and then with **LangChain**. We saw that no matter how fancy the name (Memory RAG, Adaptive RAG, Corrective RAG), the skeleton was the same:

- Ingest documents → Chunk → Embed → Store → Retrieve → Prepare Context → Prompt → Answer.
  LangChain gave us neat lego blocks to assemble this workflow: retrievers, memory, chains, agents.

But here's the question:

Is a vector database alone enough to manage all kinds of information and queries?

Think of a library. If you only use the "similar books" section (semantic vectors), you might miss:

- a book that exactly matches a keyword you typed,
- a summary that explains the gist without flipping through 200 pages,
- or a neat hierarchy where topics are grouped under chapters and sections.

This is where **LlamaIndex steps in.**

1. What is LlamaIndex

LlamaIndex is like a **Swiss Army knife for connecting LLMs to your data.**
Where LangChain focuses on *orchestration* (how pieces flow together), LlamaIndex focuses on *knowledge indexing* (how your data is stored, structured, and retrieved smartly).

Imagine you upload PDFs, scrape web pages, and store text in a vector DB. With vanilla retrieval, the LLM only sees "closest passages." But LlamaIndex says:

- "Hey, sometimes you need a summary of the whole doc, not just a slice."
- "Sometimes keywords are the best way to catch details."
- "Sometimes building a hierarchy (like table of contents → section → paragraph) makes the query easier to answer."

It's like adding **extra brains on top of your data**, beyond raw vectors.

1.1 Core abstractions in LlamaIndex

Instead of drowning in APIs, let's look at the **mental model**:

- **Document → Node**
  - A document is your PDF, web page, or text file.
  - LlamaIndex breaks it into *nodes* — smaller, manageable chunks with metadata.
  - Think of "nodes" like flashcards created out of a big book.
- **Index**
  - Once you have nodes, you build an **index**.
  - An index is not just vectors — it can be:
    - **VectorStoreIndex** → semantic similarity (like Pinecone).
    - **KeywordTableIndex** → a dictionary of keywords.
    - **SummaryIndex** → pre-computed summaries.
    - **TreeIndex** → hierarchy (chapter → section → para).
  - Each index offers a *different lens* to look at the same data.

- **Query Engine**
  - Instead of manually stitching retriever + reranker + prompt, LlamaIndex gives you a *query engine*.
  - You send a query, it decides: retrieve from which index, combine results, synthesize an answer.
  - It's like an intelligent librarian who not only fetches the books but also summarizes them for you.

1.2 How it goes beyond "vector only" retrieval

Why not just rely on vector search?

- Vectors are great for *semantic matches* ("What drink is calming?" → pulls Ashwagandha coffee).
- But vectors sometimes miss:
  - **Exact keywords** (like a date, ratio, or chemical formula).
  - **Global summaries** (e.g., "What is this whole PDF about?").
  - **Hierarchical navigation** (e.g., "In Chapter 3, what are the side effects of turmeric coffee?").

LlamaIndex plugs these gaps:

- Keyword index = catches exact matches.
- Summary index = lets you "zoom out."
- Tree index = lets you drill down step by step.

So instead of only "finding passages," LlamaIndex lets the LLM **choose the right view of the data**.

1.3 What about Langchains

LangChain *can* handle multiple retrievers (dense, sparse, hybrid). You can stitch them into an EnsembleRetriever, add rerankers, etc.
The difference is in **philosophy**:

- **LangChain = Flow manager.** It gives you the *pipes* to connect models, tools, retrievers.
- **LlamaIndex = Knowledge manager.** It gives you *structured indexes* and *query engines* that are ready-to-use.

Think of LangChain as your **plumbing system** — it decides how water flows.
LlamaIndex is the **water tank + filters** — it decides how the water is stored, purified, and

accessed.They overlap, yes. But the emphasis is different. That's why in many real projects, people actually combine both.

Think of it like this-

- A vector DB alone is like a music playlist sorted only by "vibe." Good, but limited.
- LlamaIndex lets you sort by vibe (vector), exact lyrics (keywords), artist summaries (summary index), or even album hierarchy (tree index).
- LangChain then decides how these retrievals flow into a bigger conversation system.

1.4 Understanding the basic structure of LlamaIndex

Retrieval and Querying using LlamaIndex

```
import os
from pinecone import Pinecone
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.core import VectorStoreIndex
from llama_index.llms.gemini import Gemini
from llama_index.embeddings.gemini import GeminiEmbedding



# --- Connect to Pinecone ---
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index("coffee-hybrid")   # replace with your index name

# --- Wrap Pinecone VectorStore ---
li_store = PineconeVectorStore(pinecone_index=index)

# --- Gemini Embeddings (must match what you used for upserts) ---
embed = GeminiEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ["GEMINI_API_KEY"]
)

# --- Build LlamaIndex wrapper ---
index_li = VectorStoreIndex.from_vector_store(li_store,
```

```
embed_model=embed)

# --- Gemini LLM ---
li_llm = Gemini(
    model="models/gemini-1.5-flash",
    api_key=os.environ["GEMINI_API_KEY"]
)

# --- Query Engine ---
qe = index_li.as_query_engine(llm=li_llm)
print(qe.query("Explain desi brewing."))
```

Think of LlamaIndex as having **3 main building blocks**:

Block 1 - Data Layer (VectorStore / Index)

- Where your documents live (e.g., Pinecone, Weaviate, FAISS, in-memory).
- You wrapped Pinecone with:

```
li_store = PineconeVectorStore(pinecone_index=index)
index_li = VectorStoreIndex.from_vector_store(li_store,
embed_model=embed)
```

So here:

- `li_store` = adapter that lets LlamaIndex talk to Pinecone.
- `index_li` = a LlamaIndex abstraction that understands how to **embed queries** and **retrieve matching vectors** from Pinecone.

Block 2 - Model Layer (LLM + Embeddings)

- **Embeddings** are used for retrieval:

```
embed = GeminiEmbedding(model_name="models/text-embedding-004")
```

- **LLM** is used for generation:
  `li_llm = Gemini(model="models/gemini-1.5-flash")`
  This takes the retrieved docs and your query, then produces a natural-language answer.

Block 3 - Query Layer (Query Engine / Retriever / Synthesizer)

- The **query engine** glues retrieval + generation together:
  `qe = index_li.as_query_engine(llm=li_llm)`
- Inside this engine:
  - `Retriever` = talks to the index (here Pinecone) to fetch top-K docs.
  - `Response Synthesizer` = takes retrieved docs + query → feeds them into Gemini LLM → returns final answer.

Visual Flow

```
 User Query  ──►  Query Engine
                     │
                     ├── Retriever (VectorStoreIndex)
                     │       │
                     │       └── PineconeVectorStore → Pinecone
index
                     │
                     └── Response Synthesizer
                             │
                             └── Gemini LLM (uses retrieved docs)
```

Core Building Blocks of LlamaIndex

When you hear "LlamaIndex," it might sound like just another wrapper on top of vector databases. But that's not the right way to think about it. LlamaIndex is more like a **control panel** that lets you mix different ways of storing, searching, and reasoning over data — all so your LLM can give grounded answers instead of hallucinations.

To really understand how it works, let's break it down into its **four core building blocks**: the **Index Layer**, **Embeddings**, the **LLM**, and the **Query Engine**.

1. Index Layer / Data Layer

The Index Layer is the **foundation of LlamaIndex**. If you think back to the last class, when we built a RAG pipeline by hand, the first step was always: *"Where do we put the documents?"* We chunked text, made embeddings, and stored them in Pinecone. That storage step was our "knowledge shelf."

But here's the twist: in LlamaIndex, this shelf isn't just one type. It can be built in different shapes depending on what you need later — sometimes you want detailed semantic search, sometimes you just want quick summaries, and sometimes you want exact keyword lookups. The Index Layer is the **container that makes all of that possible.**

1.1 Any Backend Store

The index doesn't care where you keep the data — it could be Pinecone, FAISS, Qdrant, Milvus, MongoDB, Postgres, or even just memory on your laptop.
Think of it like a **plug adapter**: no matter what power socket (backend) you have, the Index Layer gives you a uniform interface to plug into it.

So if tomorrow you swap Pinecone for FAISS, or Postgres for Qdrant, the rest of your code doesn't break.

```
----------------------------
import os
from pinecone import Pinecone
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader,
Settings, get_response_synthesizer
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding
from llama_index.core.retrievers import QueryFusionRetriever
from llama_index.core.query_engine import RetrieverQueryEngine

# ============================ Embedding layer
============================
# Force Google models globally BEFORE anything else (prevents OpenAI
fallback)
os.environ.pop("OPENAI_API_KEY", None)  # ensure no accidental
fallback
Settings.embed_model = GoogleGenAIEmbedding(
```

```
    model_name="models/text-embedding-004",  # MUST match your
Pinecone vectors (768-d)
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)

# ============================ Data Layer
============================
# Pinecone (existing index built with text-embedding-004)
INDEX_NAME = "coffee-hybrid"
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index(INDEX_NAME)
vstore = PineconeVectorStore(pinecone_index=pc_index)        #
optional: namespace="..."

# Local PDFs (NO Pinecone upsert; kept in-memory)
docs = SimpleDirectoryReader("./coffee_docs").load_data()

# Build indexes (use Settings.embed_model)
vector_idx = VectorStoreIndex.from_vector_store(vstore)       #
Pinecone-backed
local_idx  = VectorStoreIndex.from_documents(docs)            #
in-memory for PDFs

# ============================ Query layer
============================ (Optional for now)
# Auto-fuse Pinecone + PDFs; no query expansion (no hidden LLM calls
in retrieval)
pinecone_ret = vector_idx.as_retriever(similarity_top_k=5)
pdf_ret      = local_idx.as_retriever(similarity_top_k=5)

hybrid = QueryFusionRetriever(
    retrievers=[pinecone_ret, pdf_ret],
```

```
    similarity_top_k=5,
    mode="reciprocal_rerank",
    num_queries=1,
    use_async=False,
)

qe = RetrieverQueryEngine.from_args(
    retriever=hybrid,
    response_synthesizer=get_response_synthesizer(llm=Settings.llm,
response_mode="compact")
)

print(qe.query("Using both PDFs and the vector KB, explain desi
brewing in 3 bullets."))
```

1.2 Different Types of Indexes

This is where LlamaIndex really shows it's not just a "vector DB wrapper." It gives you multiple "views" over your data:

- **VectorStoreIndex**
  The classic one. Documents are embedded and stored in a vector DB (like Pinecone). Later, you can do semantic search: ask "What is turmeric coffee?" and the system finds chunks that *mean* turmeric coffee, even if the exact words don't match.

- **SummaryIndex**
  Instead of storing every little chunk, this index holds precomputed summaries of entire documents. Perfect when the user asks "Can you explain what this 200-page policy is about?" It jumps straight to summaries without doing token-by-token retrieval.

- **KeywordTableIndex**
  A lighter option. It builds a simple keyword table — basically a fast lookup of important words, headings, or titles. This is useful for exact term matches, like "Find all entries with the keyword *ashwagandha* in the title."

  Let's try to compare it with BM25 we have used earlier:

Feature	Keyword Table Index (LlamaIndex)	BM25
Method	Simple keyword → node mapping	Ranking function using TF–IDF + length norm
Ranking	No ranking, just retrieves matches	Provides ranked results by relevance
Weighting	None (all keywords equal)	Rare terms weighted higher, frequent terms lower
Scalability	Works best for small corpora	Handles large corpora more effectively
Integration	Built-in index within LlamaIndex	Used via retrievers (external integration)
Use Case	Quick lookups, lightweight pipelines	Strong baseline for keyword retrieval
Limitation	Poor for nuanced relevance	No semantic understanding

- **Tree-Index**
  Imagine your document structured like a table of contents. At the top, you have chapter summaries; below, section summaries; below that, paragraph details. A query can first zoom into the right branch of the tree, then go deeper as needed. This makes it efficient for large corpora where you don't want to scan every chunk.

```
import os
from llama_index.core import (
    SimpleDirectoryReader,
    Settings,
    SummaryIndex,
    TreeIndex,
    KeywordTableIndex,   # <-- correct import (core)
)
from llama_index.llms.google_genai import GoogleGenAI
```

```
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding

# ============================ Embedding / LLM layer
============================
os.environ.pop("OPENAI_API_KEY", None)  # avoid fallback
Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)

# ============================ Data layer (PDFs only)
============================
# Read ONLY PDFs from ./coffee_docs
try:
    docs = SimpleDirectoryReader(
        input_dir="./coffee_docs",
        required_exts=[".pdf"],
    ).load_data()
except TypeError:
    # fallback for older versions
    docs = SimpleDirectoryReader(
        input_dir="./coffee_docs",
        file_exts=[".pdf"],
    ).load_data()

# ============================ Build separate indexes (no fusion)
============================
summary_idx = SummaryIndex.from_documents(docs)        # precomputed
per-doc summaries
keyword_idx = KeywordTableIndex.from_documents(docs)   # exact/term
lookups
tree_idx    = TreeIndex.from_documents(docs)           # hierarchical
```

```
zoom-in

# ============================ Query layer (each used independently)
============================
# 1) SummaryIndex -- "explain the 200-page policy" style
summary_qe = summary_idx.as_query_engine(llm=Settings.llm)
print(summary_qe.query("Give a high-level summary of these PDFs in 5
bullets."))

# 2) Keyword Table Index -- fast exact/term matches
keyword_qe = keyword_idx.as_query_engine(llm=Settings.llm)
print(keyword_qe.query("Find occurrences where 'ashwagandha' appears
in titles or headings."))

# 3) Tree Index -- navigate top→down like a table of contents
tree_qe = tree_idx.as_query_engine(llm=Settings.llm)
print(tree_qe.query("Locate sections about brewing temperature and
summarize them."))
```

1.3 Why It Matters

The beauty here is **choice**. In a normal RAG pipeline, you only had one option: chunk documents, put them in Pinecone, and pray your query retrieves the right context. With LlamaIndex's Index Layer, you can pick the right "lens" for the job:

- Need **semantic meaning** → VectorStoreIndex.
- Need **high-level overviews** → SummaryIndex.
- Need **fast keyword hits** → Keyword Table.
- Need **hierarchical drill-downs** → Tree Index.

Analogy: Think of it like Google Maps. Sometimes you want the full street view (VectorStoreIndex). Sometimes you just want the high-level map overview (SummaryIndex). Sometimes you want to type the exact address (Keyword Table). Sometimes you want to zoom gradually from country → state → city → street (Tree Index). The Index Layer is what makes all of these navigation modes possible.

1.4 Concept of a node

At the heart of LlamaIndex, everything boils down to a **Node**.
Think of a Node as the **basic Lego block of knowledge**.

- A Node = a chunk of text (or data) **plus** some useful **metadata** that tells us about where it came from.
- Metadata = source (PDF filename, page number, paragraph heading), timestamp, keywords, embeddings, summaries, etc.
- Nodes can look slightly different depending on which kind of index you're using, but the idea is the same:
  Break raw documents into smaller, manageable pieces that can later be searched, summarized, or linked together.

Sticky analogy: Imagine you're making flashcards from a textbook. Each flashcard = a Node. You don't copy the whole 200-page book onto one card. Instead, you create many smaller flashcards, and each one also notes the **page number, topic heading, and keywords**.

Nodes in different types of Index:

a. VectorStoreIndex (the classic one)

- Each Node here is basically:
  A chunk of text (say, 300–500 words).
  - Its **embedding vector** (the numerical fingerprint).
  - Metadata like `{"file": "coffee.pdf", "page": 12, "section": "Turmeric"}`.
- Stored inside a **vector database** (Pinecone, FAISS, Weaviate, etc.).
- What happens:
  - When you ask "What is turmeric coffee?", LlamaIndex embeds your query into a vector.
  - It compares that query vector with all Node vectors in the DB.
  - The closest ones (semantically similar) are returned as context.
- **So the Node here is like a flashcard with a long "semantic fingerprint" added.**

b. SummaryIndex

- Here, instead of keeping every chunk as a Node, you keep **summaries as Nodes**.
- For a long document (say, 200 pages), the system might:
  - Summarize each chapter → store those summaries as Nodes.
  - Optionally, also store a "global" summary of the whole doc.
- A Node here looks like:

- A summary text: *"This policy describes data privacy measures for healthcare providers…"*
- Metadata: `{"doc": "privacy_policy.pdf", "chapter": "Data Retention", "length": 40 pages}`.

- What happens:
  - When you ask, "What is this policy about?", the system skips chunk retrieval and just serves you the pre-made summaries.
- **So the Node here is like a flashcard where you wrote a *summary* instead of the full text.**

c. Keyword Table Index

- Think of this as a Node designed for **fast keyword lookups.**
- Instead of fancy embeddings, you store:
  - A keyword list (extracted from the doc).
  - Pointers to where those keywords appear (file, heading, page).
- A Node here looks like:

```
{
  "keywords": ["ashwagandha", "adaptogen", "latte"],
  "source": "herbal_coffee.html",
  "chunk": "Ashwagandha latte is known for calming effects..."
}
```

- What happens:
  - If you ask, "Find me all entries with ashwagandha in the title," the system just consults the keyword table.
  - No embeddings or semantic search required.
- **So the Node here is like an index card where you only bolded and stored the important keywords.**

c. Tree Index

- This one is hierarchical — Nodes are linked in a **parent–child tree structure.**
- Example: You feed a 200-page book. LlamaIndex will:
  - Create Nodes for each paragraph.
  - Summarize those into section-level Nodes.
  - Summarize again into chapter-level Nodes.
  - Finally, one Node at the top represents the whole book.
- A Node here can therefore look like:

```
          {
  "summary": "Chapter 5 explains turmeric and saffron coffee
benefits...",
  "children": [Node_51, Node_52, Node_53],
  "level": "chapter",
  "source": "coffee_book.pdf"
}
```

Code to sneak peak into nodes

```
# ---------------------------- IMPORTS (all in one place)
----------------------------
from textwrap import shorten
from llama_index.core import (
    Settings,
    Document,
    VectorStoreIndex,
    SummaryIndex,
    KeywordTableIndex,
    TreeIndex,
)
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

# ============================ Embedding layer (no API calls)
============================
Settings.embed_model =
HuggingFaceEmbedding("sentence-transformers/all-MiniLM-L6-v2")
Settings.llm = None  # ensure NO generation; we'll only retrieve
Nodes

# ============================ Data (sample texts → Documents; no
manual Nodes) ============================
docs = [
    Document(text="Turmeric coffee (golden latte) blends coffee with
turmeric and spices; earthy and slightly peppery."),
    Document(text="Ashwagandha often appears in wellness menus; exact
keyword lookups help when scanning titles and headings."),
```

```
    Document(text="Brewing temperature: lighter roasts typically
92-96°C; hotter extractions risk bitterness."),
    Document(text="Espresso uses fine grind, ~9 bar pressure, and
short contact time to create a concentrated cup."),
]

# ============================ Index build (LlamaIndex will create
Nodes internally) ============================
vec_index   = VectorStoreIndex.from_documents(docs,
transformations=[])  # semantic search
sum_index   = SummaryIndex.from_documents(docs, transformations=[])
# precomputed summaries
kw_index    = KeywordTableIndex.from_documents(docs,
transformations=[]) # keyword table
tree_index  = TreeIndex.from_documents(docs, transformations=[])
# hierarchical

# ============================ "Show me a Node" -- WITHOUT creating
one explicitly ============================
def peek_one_node(index, query: str):
    hit = index.as_retriever(similarity_top_k=1).retrieve(query)[0]
    node = hit.node
    node_id = getattr(node, "node_id", getattr(node, "id_", None))
    print(f"\n[{index.__class__.__name__}] query='{query}'")
    print("node_id :", node_id)
    print("text    :", shorten(node.get_content(), width=140,
placeholder=" ..."))
    print("metadata:", dict(list(node.metadata.items())[:4]))
    rels = getattr(node, "relationships", {})
    print("rels    :", list(rels.keys()) if rels else "--")

# VectorStoreIndex → returns the embedded Node that matched
semantically
peek_one_node(vec_index, "What is turmeric coffee?")

# SummaryIndex → returns the summary Node it stored for the doc
peek_one_node(sum_index, "Explain turmeric coffee in brief")
```

```
# KeywordTableIndex → returns a Node found via exact term lookup
peek_one_node(kw_index, "ashwagandha")

# TreeIndex → returns a Node from the most relevant branch
(top→down)
peek_one_node(tree_index, "brewing temperature")
```

2. Embeddings & LLM Layer

If the **Index Layer** is like the *shelf* where you store knowledge, the **Embeddings & LLM Layer** is the *translator + storyteller*.

- The **embedding model** is the translator. It takes human language (words, sentences, PDFs) and turns them into numerical fingerprints (vectors) that machines can understand and compare.
- The **LLM** is the storyteller. It takes those retrieved vectors, reads the actual text behind them, and crafts a human-sounding answer.

Both are equally important. A bad embedding model means you won't retrieve the right chunks (garbage in, garbage out). A bad LLM means you may retrieve the right chunks but get clumsy or hallucinated answers.

2.1 Embeddings

Embeddings are **the backbone of retrieval.** Without them, your index can't "understand meaning."

- **What they do**
  - Convert text → vector (a list of floating-point numbers, usually 300–1,000 dimensions).
  - Similar texts have vectors that sit close together in this high-dimensional space.
  - Example: "turmeric coffee" and "golden latte" → embeddings will be neighbors, even though the words differ.

- **Which embedding models can you plug in?**
  - OpenAI (text-embedding-3-small, text-embedding-3-large)
    Very strong, production-ready, but paid + external API.
  - Gemini / Google (models/text-embedding-004)
    Newest, optimized for Google's ecosystem, 768-dimensional vectors.

- Cohere (embed-english-v3, embed-multilingual-v3)
  Good multilingual coverage, great for enterprise search.
- HuggingFace Sentence Transformers (MiniLM, all-mpnet-base-v2, etc.)
  Free + local, but usually less powerful than OpenAI/Gemini at scale.
- Local Models (via llama.cpp, Ollama, etc.)
  For privacy-sensitive setups. No API calls, but heavier infra needed.

- **Settings you control**
  - Dimension → Must match the vector DB index (e.g., Gemini = 768, OpenAI = 1,536).
  - Granularity → Do you embed whole paragraphs, sentences, or smaller chunks? Smaller = more precise, larger = more context.
  - Multi-modal → You can also embed images, audio, tables alongside text (if your model supports it). This is how LlamaIndex can handle PDFs with figures, or multi-media corpora.
- Embeddings are like the "GPS coordinates" of your documents. Different GPS systems (OpenAI vs Gemini vs HuggingFace) give slightly different maps, but they all let you locate where "turmeric coffee" lives in the knowledge space.

Code to show how LlamaIndex organizes embeddings

```
import os
from llama_index.core import (
    SimpleDirectoryReader,
    Settings,
    SummaryIndex,
    TreeIndex,
    KeywordTableIndex,
)
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
# <-- works after plugin install

# ============================== Embedding layer (no API calls)
==============================
# Small, fast local model (384-d). You can swap to
"all-mpnet-base-v2" (768-d) if you prefer.
Settings.embed_model =
HuggingFaceEmbedding(model_name="sentence-transformers/all-MiniLM-L6-
v2")
```

```
from llama_index.core import Settings
Settings.llm = None  # hard-disable any LLM usage


# ============================ Data layer (PDFs only)
============================
try:
    docs = SimpleDirectoryReader(input_dir="./coffee_docs",
required_exts=[".pdf"]).load_data()
except TypeError:
    docs = SimpleDirectoryReader(input_dir="./coffee_docs",
file_exts=[".pdf"]).load_data()

# Build each index locally (no Pinecone, no fusion)
summary_idx = SummaryIndex.from_documents(docs)       # precomputed
per-doc summaries
keyword_idx = KeywordTableIndex.from_documents(docs)  # exact/term
lookups
tree_idx    = TreeIndex.from_documents(docs)          # hierarchical
zoom-in

# ============================ Query layer (retrieval-only; no LLM
spend) ============================
def preview_top_nodes(index, query: str, k: int = 3):
    hits = index.as_retriever(similarity_top_k=k).retrieve(query)
    print(f"\nTop {k} for: {query}")
    for h in hits:
        print("-" * 70)
        print(h.node.get_content()[:700], "...")

preview_top_nodes(summary_idx, "High-level summary of these PDFs")
preview_top_nodes(keyword_idx, "ashwagandha in titles or headings")
preview_top_nodes(tree_idx,    "brewing temperature guidance")
```

How chunking works (quick)

- **Normal chunking (Sentence/Token splitter)**
  Splits by length with optional overlap. Fast, deterministic. Good defaults:
  - `SentenceSplitter(chunk_size=512, chunk_overlap=64)` (token-ish units)
  - (alt) `TokenTextSplitter(...)` if you want pure token windows.
- **Semantic chunking**
  Computes embeddings over sliding windows and places **breakpoints** where content meaning shifts (e.g., heading → paragraph). More coherent chunks, fewer mid-sentence cuts. Tuned by:
  - `breakpoint_percentile_threshold` (lower = more breaks; higher = fewer)
  - `buffer_size` (how many sentences to consider around potential breaks)

```
# NORMAL CHUNKING (sentence-based)
# pip install -U llama-index-core llama-index-embeddings-huggingface
sentence-transformers pypdf

from llama_index.core import SimpleDirectoryReader, Settings,
VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

Settings.embed_model =
HuggingFaceEmbedding("sentence-transformers/all-MiniLM-L6-v2")
Settings.llm = None  # no generation

docs  = SimpleDirectoryReader("./coffee_docs",
required_exts=[".pdf"]).load_data()
nodes = SentenceSplitter(chunk_size=512,
chunk_overlap=64).get_nodes_from_documents(docs)
index = VectorStoreIndex.from_nodes(nodes)

hits = index.as_retriever(similarity_top_k=3).retrieve("brewing
temperature")
for h in hits:
    print("-" * 70, "\n", h.node.get_content()[:700], "...")
```

```
# SEMANTIC CHUNKING (meaning-aware splits)
# pip install -U llama-index-core llama-index-embeddings-huggingface
sentence-transformers pypdf

from llama_index.core import SimpleDirectoryReader, Settings,
VectorStoreIndex
from llama_index.core.node_parser import SemanticSplitterNodeParser
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

Settings.embed_model =
HuggingFaceEmbedding("sentence-transformers/all-MiniLM-L6-v2")
Settings.llm = None  # no generation

docs  = SimpleDirectoryReader("./coffee_docs",
required_exts=[".pdf"]).load_data()
sem   = SemanticSplitterNodeParser(
    embed_model=Settings.embed_model,
    buffer_size=3,
    breakpoint_percentile_threshold=95,
)
nodes = sem.get_nodes_from_documents(docs)
index = VectorStoreIndex.from_nodes(nodes)

hits = index.as_retriever(similarity_top_k=3).retrieve("brewing
temperature")
for h in hits:
    print("-" * 70, "\n", h.node.get_content()[:700], "...")
```

2.2 LLM

Once retrieval is done, we need someone to actually **read the docs and answer the question.** That's where the LLM comes in.

- What they do
    - Read the prompt (user question + retrieved context).
    - Generate a fluent answer, summary, or reasoning.

    - Can also generate structured outputs (JSON, tables, bullet points) depending on the prompt.
- Which LLMs can you plug in?
    - Gemini (Google) → gemini-1.5-flash (fast, cheaper), gemini-1.5-pro (slower, smarter).
    - OpenAI GPT models → gpt-4o, gpt-4-turbo, gpt-3.5.
    - Anthropic Claude → Very strong at reasoning + safety.
    - Mistral / Llama (open source) → Run locally if infra is ready.
    - Mixtral, Phi, etc. → lightweight models, great for cost-sensitive use cases.
- Settings you control
    - Temperature → Controls creativity.
        - 0.0 = deterministic, great for factual Q&A.
        - 0.7+ = more creative, great for brainstorming.
    - Max Tokens → Cap on answer length.
        - Helps avoid long rambles, or fit within UI constraints.
    - Response Format → Plain text vs JSON vs bullet points.
        - LlamaIndex lets you enforce this with output parsers.
    - System Prompts → Define the personality of the assistant (polite, strict, research-style, etc.).

If embeddings are GPS coordinates, the LLM is your **tour guide**. You point it to the right neighborhood (retrieval), but the guide decides how to describe what's there: short summary, long lecture, or even a table of pros/cons.

a. Why They Work Together

- If your embeddings are weak, the LLM is like a guide who gets dropped in the wrong city — no matter how good the guide is, they'll describe the wrong place.
- If your LLM is weak, it's like dropping the bad guide into the right city but they can't explain well — they'll just babble facts or make mistakes.

That's why the **Embeddings + LLM layer is always designed as a pair.** You choose both based on:

- **Budget** (OpenAI/Gemini vs HuggingFace local).
- **Use case** (factual answers vs creative storytelling).
- **Privacy** (cloud API vs local deployment).

b. Simple Implementation

```
import os
from pinecone import Pinecone
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.core import VectorStoreIndex, Settings
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding

# ============================ Embedding / LLM layer (define via
Settings) ============================
# We previously set Settings.llm = None; set them explicitly here.
# Must match your Pinecone vectors: models/text-embedding-004 →
768-d
Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)

# ============================ Data / Index layer (Pinecone only)
============================
INDEX_NAME = "coffee-hybrid"   # existing Pinecone index built with
004
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index(INDEX_NAME)

# If your text was stored under a different key (e.g.,
"page_content"), change text_key.
vstore = PineconeVectorStore(pinecone_index=pc_index,
text_key="text")  # optional: namespace="..."
index  = VectorStoreIndex.from_vector_store(vstore)  # uses
Settings.embed_model
```

```
# ============================ Query layer (retriever + LLM)
============================
qe = index.as_query_engine(similarity_top_k=5)  # uses Settings.llm
print(qe.query("What is turmeric coffee?"))
```

3. Query Layer

If the **Index Layer** is your library (all the books, pages, summaries, and keywords neatly stored as Nodes), then the **Query Layer** is the librarian.

It's the part that takes your messy human question — *"Hey, what's turmeric coffee and is it good for digestion?"* — and figures out **how to talk to the Index, what to pull out, and how to shape that into a useful response**.

3.1 What the Query Layer *is*

- It's the **orchestrator** sitting on top of your Indexes (VectorStoreIndex, SummaryIndex, TreeIndex, etc.).
- Instead of you writing manual logic ("first search Pinecone, then re-rank, then combine"), LlamaIndex gives you **Query Engines** that do this automatically, but can still be customized.
- In plain terms:
  Index = where knowledge lives.
- Query Layer = how you ask questions and get back answers.

3.2 What the Query Layer can include

When you build a query pipeline, you can mix and match components inside the Query Layer. Some key ones:

a. Retrieval strategies

- **Semantic retrieval** → Pull top-k Nodes via embedding similarity (VectorStoreIndex).
- **Keyword lookup** → Match terms or metadata (Keyword Table).
- **Hybrid retrieval** → Blend semantic and keyword signals.
- **Filtered retrieval** → Apply metadata filters (e.g., *only from 2023 docs, only from "recipes" category*).

```
import os
from pinecone import Pinecone
from llama_index.core import (
    Settings,
    SimpleDirectoryReader,
    VectorStoreIndex,
    get_response_synthesizer,
)
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding
from llama_index.core import KeywordTableIndex
from llama_index.core.retrievers import QueryFusionRetriever
from llama_index.core.query_engine import RetrieverQueryEngine

# Optional: instrumentation to count LLM calls (version-safe)
from llama_index.core.callbacks import CallbackManager
try:
    from llama_index.core.callbacks import TokenCountingHandler
    counter = TokenCountingHandler()
    Settings.callback_manager = CallbackManager([counter])
except Exception:
    counter = None

# ============================ 1) TURN OFF LLM WHILE LOADING OFFLINE
DOCS ============================
# This prevents any accidental LLM usage during PDF loading / keyword
index build.
Settings.llm = None

# Load PDFs locally (NO LLM CALLS here)
try:
    docs = SimpleDirectoryReader("./coffee_docs",
required_exts=[".pdf"]).load_data()
except TypeError:
    docs = SimpleDirectoryReader("./coffee_docs",
file_exts=[".pdf"]).load_data()
```

```
# Build a pure keyword lookup index from the PDFs (NO LLM CALLS)
kw_idx = KeywordTableIndex.from_documents(docs, transformations=[])

# ============================ 2) ENABLE EMBEDDING + LLM FOR QUERIES
============================
# Must match Pinecone's embedding space (you indexed with 004 /
768-d)
Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)

# Single-shot, no tools/AFC (kwargs ignored if unsupported--safe to
include)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
    temperature=0.2,
    max_tokens=512,
    tools=[],
    tool_config={"function_calling_config": "NONE",
"max_remote_calls": 0},
)

# ============================ 3) DATA / INDEX (Pinecone semantic
side) ============================
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index("coffee-hybrid")
vstore   = PineconeVectorStore(pinecone_index=pc_index,
text_key="text")  # change if you used "page_content"
sem_idx  = VectorStoreIndex.from_vector_store(vstore)  # wraps
Pinecone

# ============================ 4) QUERY LAYER -- HYBRID, SINGLE-SHOT
============================
```

```
# Retrievers (no LLM calls; Pinecone will do one 004 embed per query)
sem_ret = sem_idx.as_retriever(similarity_top_k=5)
kw_ret  = kw_idx.as_retriever(similarity_top_k=5)

# Hybrid retrieval with NO query expansion (prevents extra LLM work)
hybrid = QueryFusionRetriever(
    retrievers=[sem_ret, kw_ret],
    similarity_top_k=5,
    mode="reciprocal_rerank",
    num_queries=1,
    use_async=False,
)

# Force exactly ONE LLM generation per query
synth = get_response_synthesizer(llm=Settings.llm,
response_mode="compact")

qe = RetrieverQueryEngine.from_args(
    retriever=hybrid,
    response_synthesizer=synth,
)

def run_one(q: str):
    if counter: counter.reset_counts()
    ans = qe.query(q)
    print(ans)
    if counter:
        print("LLM calls for this query:", getattr(counter,
"total_llm_calls", "N/A"))  # should be 1

# ============================ 5) RUN QUERIES (each => exactly ONE
Gemini call) ============================
run_one("What is turmeric coffee?")
run_one("Find entries mentioning ashwagandha in titles or headings.")
run_one("Give ideal brewing temperature guidance in one short
paragraph.")
```

b. Query Transformation

Before even hitting the index, you can modify the query:

- **MultiQuery** → Expand into multiple paraphrases for broader recall.
- **SelfQuery** → Let the LLM infer metadata filters from the query ("latest", "only Indian recipes").
- **Decomposition** → Break a complex query into sub-questions (*"compare turmeric vs saffron coffee"* → two separate queries, then merged).

```
# ---------------------------- IMPORTS (all in one place)
----------------------------
import os
from pinecone import Pinecone
from llama_index.core import Settings, VectorStoreIndex,
get_response_synthesizer
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding
from llama_index.core.retrievers import QueryFusionRetriever
from llama_index.core.query_engine import RetrieverQueryEngine
from llama_index.core.vector_stores.types import MetadataFilters,
ExactMatchFilter

# =============== 1) EMBEDDING / LLM LAYER (match your Pinecone dims:
004 → 768) ===============
Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
    temperature=0.2,
    max_tokens=512,
)
```

```
# ========================== 2) DATA / INDEX LAYER (Pinecone only)
==========================
INDEX_NAME  = "coffee-hybrid"
TEXT_KEY    = "text"       # change to "page_content" if you stored
text under that
HEADING_KEY = "heading"    # or "headings" if that's your metadata
key

pc       = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index(INDEX_NAME)
vstore   = PineconeVectorStore(pinecone_index=pc_index,
text_key=TEXT_KEY)
index    = VectorStoreIndex.from_vector_store(vstore)  # uses
Settings.embed_model

# ================================ 3) QUERY LAYER
================================
# --- A) MultiQuery: expand into paraphrases, fuse, single-shot
synthesis ---
def rag_multiquery(question: str):
    sem_ret = index.as_retriever(similarity_top_k=5)
    fused = QueryFusionRetriever(
        retrievers=[sem_ret],
        similarity_top_k=5,
        mode="reciprocal_rerank",
        num_queries=4,          # LLM creates ~3 paraphrases +
original
        use_async=False,
    )
    synth = get_response_synthesizer(llm=Settings.llm,
response_mode="compact")  # single LLM call to answer
    qe = RetrieverQueryEngine.from_args(retriever=fused,
response_synthesizer=synth)
    return qe.query(question)

# --- B) SelfQuery (headings): infer a heading term → filter by
Pinecone metadata, then answer ---
```

```
def rag_selfquery_by_heading(question: str):
    # Ask LLM for a concise heading/section term (plain text, minimal
cost)
    prompt = f"From this question, guess the most relevant document
heading in 2-4 words (no quotes):\n{question}"
    heading_term = Settings.llm.complete(prompt).text.strip()

    filters =
MetadataFilters(filters=[ExactMatchFilter(key=HEADING_KEY,
value=heading_term)])
    ret    = index.as_retriever(similarity_top_k=5, filters=filters)
    synth = get_response_synthesizer(llm=Settings.llm,
response_mode="compact")
    qe     = RetrieverQueryEngine.from_args(retriever=ret,
response_synthesizer=synth)
    return qe.query(question)

# ---------------------------- EXAMPLES ----------------------------
print(rag_multiquery("What is turmeric coffee?"))
print(rag_selfquery_by_heading("Ashwagandha about it and benefits?"))
```

3.3 Routing

Sometimes one query doesn't fit all. Query Layer can:

- **RouterQueryEngine** → Send different queries to different indexes.
    - E.g., health-related → medical index, recipes → cooking index.
- **SubQuestionQueryEngine** → Break query into parts, send each to the right index, then merge.

```
import os
from pinecone import Pinecone
from llama_index.core import (
    Settings,
    VectorStoreIndex,
    SimpleDirectoryReader,
    KeywordTableIndex,
```

```
)
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding
from llama_index.core.tools import QueryEngineTool, ToolMetadata
from llama_index.core.selectors import LLMSingleSelector
from llama_index.core.query_engine import RouterQueryEngine,
SubQuestionQueryEngine

# ============================ 1) EMBEDDING / LLM LAYER
============================


# ============================ 2) DATA / INDEX LAYER
============================
# Keep LLM OFF while loading PDFs (prevents accidental calls)
Settings.llm = None
try:
    docs = SimpleDirectoryReader("./coffee_docs",
required_exts=[".pdf"]).load_data()
except TypeError:
    docs = SimpleDirectoryReader("./coffee_docs",
file_exts=[".pdf"]).load_data()
kw_idx = KeywordTableIndex.from_documents(docs)

# Pinecone semantic index
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index("coffee-hybrid")
vstore = PineconeVectorStore(pinecone_index=pc_index,
text_key="text")  # change if you used "page_content"
sem_idx = VectorStoreIndex.from_vector_store(vstore)

# Turn LLM ON now
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
    temperature=0.2,
```

```
    max_tokens=512,
)

Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",  # must match Pinecone
vectors (768-d)
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)

# Wrap each index as a query engine (single-shot synthesis via
response_mode="compact")
qe_sem = sem_idx.as_query_engine(similarity_top_k=5,
response_mode="compact", llm=Settings.llm)
qe_kw  = kw_idx.as_query_engine(response_mode="compact",
llm=Settings.llm)

# Package as proper tools (use ToolMetadata, not dict)
tools = [
    QueryEngineTool(query_engine=qe_sem,
metadata=ToolMetadata(name="semantic", description="General coffee KB
(Pinecone)")),
    QueryEngineTool(query_engine=qe_kw,
metadata=ToolMetadata(name="keyword",  description="PDF
keyword/headings lookup")),
]

# ============================ 3) QUERY LAYER
============================
# (A) RouterQueryEngine -- route whole query to the best engine
router = RouterQueryEngine.from_defaults(
    query_engine_tools=tools,
    selector=LLMSingleSelector.from_defaults(llm=Settings.llm),
)
print(router.query("Find entries mentioning ashwagandha in titles or
headings."))  # likely routes to keyword
print(router.query("What is turmeric coffee?"))
# likely routes to semantic
```

```
# (B) SubQuestionQueryEngine -- split, route parts, then merge
subq = SubQuestionQueryEngine.from_defaults(
    query_engine_tools=tools,
    llm=Settings.llm,
)
print(subq.query("Compare turmeric coffee vs saffron coffee and give
a one-paragraph verdict."))
```

3.4 Post-retrieval processing

After pulling candidates:

- **Re-ranking** → Use a cross-encoder or LLM to reorder results by relevance.
- **Deduplication** → Drop near-duplicate Nodes (same chunk repeated).
- **Compression** → Shrink long chunks into query-focused summaries (fit more into the context window).
- **Context budgeting** → Pack the right number of tokens into the prompt, so the LLM doesn't get overloaded.

```
# ============================ 3) Query layer (Rerank + Dedup +
Compress + Budget) ============================
def answer_with_controls(question: str,
                         base_k: int = 20,
                         rerank_top_n: int = 6,
                         dedup_cutoff: float = 0.90,
                         char_budget: int = 3500):
    """
    Reranking: cross-encoder reorders by relevance
    Deduplication: drops near-duplicate nodes
    Compression: LLM extracts query-focused snippets
    Context budgeting: packs <= char_budget into the prompt
    """

    # --- 3.1 Retrieve (semantic) ---
    retriever = index.as_retriever(similarity_top_k=base_k)
    nodes = retriever.retrieve(question)   # NO LLM calls, but 1
```

```
embed call/query (004) for Pinecone

    # --- 3.2 RERANK (cross-encoder) ---
    # Uses a cross-encoder from sentence-transformers (one local
forward pass per pair)
    reranker = SentenceTransformerRerank(
        model="cross-encoder/ms-marco-MiniLM-L-6-v2",  # light + good
        top_n=rerank_top_n,
    )
    nodes = reranker.postprocess_nodes(nodes, query_str=question)

    # --- 3.3 DEDUP (near-duplicate removal) ---
    # Drops nodes too similar to previously kept ones
    dedup = SimilarityPostprocessor(similarity_cutoff=dedup_cutoff)
    nodes = dedup.postprocess_nodes(nodes)

    # --- 3.4 COMPRESSION (LLM) ---
    # Extract the most relevant spans to shrink long chunks (THIS
uses 1 LLM call per node kept)
    compressor = LLMChainExtractor.from_defaults(llm=Settings.llm)
    nodes = compressor.postprocess_nodes(nodes, query_str=question)

    # --- 3.5 CONTEXT BUDGETING (simple char-budget packer) ---
    context_parts, total = [], 0
    for n in nodes:
        t = n.node.get_content()
        if total + len(t) > char_budget:
            t = t[: max(0, char_budget - total)]
        if t:
            context_parts.append(t)
            total += len(t)
        if total >= char_budget:
            break
    context = "\n\n---\n\n".join(context_parts)

    # --- 3.6 FINAL ONE-SHOT SYNTHESIS (exactly 1 LLM call) ---
    prompt = f"""Answer the QUESTION using only the CONTEXT. Be
concise and cite sections briefly if helpful.
```

```
CONTEXT:
{context}

QUESTION:
{question}
"""
    return Settings.llm.complete(prompt).text
```

3.5 Prompt templating with LlamaIndex

In LlamaIndex, "prompting" is about shaping how the LLM thinks **during answer synthesis**, not about re-implementing retrieval. You control the *voice*, *structure*, *citations*, and *length* of the final response by giving the query engine explicit templates.

The most common entry point is the **Text QA template** (used on the first pass) and the **Refine template** (used when the engine incrementally adds more evidence). LlamaIndex always injects two variables into your template: `context_str` (the stitched text from retrieved nodes) and `query_str` (the user's question).

You write exactly how the model should use those. A practical pattern is: state the role, forbid outside knowledge, require citations from the context, define output style (bullets/JSON), and add brevity rules.

A few practical tips that make templates "production-sticky":

- **Be explicit about grounding.** Say "Use ONLY the CONTEXT" and give a clear fallback sentence for missing info. This single line cuts hallucinations dramatically.
- **Define citation style and placement.** If you want inline `[S#]` after each claim, say so. The engine passes back `response.source_nodes` so you can map your `[S1]`… to titles/URLs in your UI.
- **Choose a response mode that matches task shape.**
    - `compact` asks the model to synthesize once from the best evidence (good for short answers).
    - `refine` streams nodes and lets the model update a draft (good when you retrieve many small, varied chunks).

- `tree_summarize` summarizes groups of nodes hierarchically (great for long "overview/explain" tasks).

- **Request structure directly in the template.** If you need JSON, ask for it and list field names. (For strict schemas, pair with an output parser or model-side JSON mode.)

- **Keep prompts short and procedural.** Concrete rules ("4–6 sentences", "no outside knowledge") beat flowery style instructions.
- **Separate roles.** Retrieval choices (k, filters, rerank) live in the query engine or post-processors; the prompt should focus on *how to use the given context*, not *how to search*.

```
import os, textwrap
from pinecone import Pinecone
from llama_index.core import Settings, VectorStoreIndex
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding

# ============================ 1) EMBEDDING / LLM LAYER
============================
# Match Pinecone vectors: text-embedding-004 → 768-d
Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
    temperature=0.2, max_tokens=600,
)

# ============================ 2) DATA / INDEX LAYER (Pinecone)
============================
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index("coffee-hybrid")
vstore   = PineconeVectorStore(pinecone_index=pc_index,
```

```
text_key="text")   # change if you used "page_content"
index    = VectorStoreIndex.from_vector_store(vstore)
# uses Settings.embed_model

# ============================ 3) QUERY LAYER (prompt templating
AFTER retrieval) ============================
def ask_with_grounded_prompt(question: str, k: int = 5,
per_source_chars: int = 900):
    """
    - Retrieval first (no LLM): top-k semantic results from Pinecone
    - Then ONE LLM call with a strict, grounded prompt:
        * "Use ONLY the CONTEXT"
        * clear fallback: 'No support in retrieved context.'
        * inline citations [S#] after claims
    - Returns (answer_text, sources_map) where sources_map is {S#:
{heading, source/url}}
    """
    # 3.1 Retrieve nodes (semantic); this triggers ONE embedding call
for the query (004)
    nodes = index.as_retriever(similarity_top_k=k).retrieve(question)

    # 3.2 Build enumerated CONTEXT and source map for your UI
    parts, sources_map = [], {}
    for i, h in enumerate(nodes, start=1):
        tag = f"S{i}"
        md  = h.node.metadata or {}
        heading = md.get("heading") or md.get("title") or "Untitled"
        src     = md.get("url") or md.get("source") or
md.get("file_path") or md.get("doc_id") or "N/A"
        snippet = h.node.get_content()
        if per_source_chars:
            snippet = snippet[:per_source_chars]
        parts.append(f"[{tag}] {heading}\n{snippet}")
        sources_map[tag] = {"heading": heading, "source": src}
    context = "\n\n---\n\n".join(parts) if parts else "(no results)"

    # 3.3 Strict, short, procedural template
(hallucination-resistant)
```

```
    prompt = f"""
You are a concise specialty-coffee expert.

RULES:
- Use ONLY the CONTEXT below.
- If the answer is not fully supported by the CONTEXT, reply exactly:
"No support in retrieved context."
- Write 4-6 sentences max.
- After each factual claim, add an inline citation like [S1], [S2]
that matches the CONTEXT tags.
- Do NOT use outside knowledge. No flowery language.

QUESTION:
{question}

CONTEXT (numbered sources):
{context}

ANSWER (with inline [S#] citations):
""".strip()

    # 3.4 One-shot synthesis (exactly ONE Gemini generation)
    answer = Settings.llm.complete(prompt).text.strip()
    return answer, sources_map


# ---------------------------- EXAMPLE ----------------------------
ans, srcs = ask_with_grounded_prompt("What is turmeric coffee and
ideal brew temperature?")
print("\n=== ANSWER ===\n", textwrap.fill(ans, 100))
print("\n=== SOURCES MAP (for UI) ===")
for tag, meta in srcs.items():
    print(f"{tag}: {meta['heading']}  |  {meta['source']}")
```

LlamaIndex vs LangChain

1. Understanding the Difference

LlamaIndex

- Think of it as a **librarian who organizes books** for you.
- You give it PDFs, Word docs, HTML pages — it chops them into chunks, attaches tags, makes summaries, and files them neatly into a cabinet (the index).
- Later, when you ask a question, it knows **exactly which drawer to open** and which pages to pull out.
- It's mostly focused on **getting the right knowledge to the LLM** in the cleanest way possible.

LangChain

- Think of it as a **personal assistant who can use many tools**.
- Sometimes it Googles, sometimes it looks in a database, sometimes it calls a calculator, sometimes it reads the library catalog (maybe from LlamaIndex).
- It's good at **planning steps**:
  "Step 1: search in Wikipedia → Step 2: do some math → Step 3: answer user."
- It's less about storing knowledge, more about **deciding the flow** of how to solve your query.

Aspect	LlamaIndex	LangChain
Core Philosophy	Purpose-built for data → LLM pipelines (retrieval, indexing, query engines). Focuses on how documents are chunked, stored, retrieved, summarized, and structured for RAG.	General LLM orchestration framework. Focuses on how LLMs talk to tools, APIs, memory, and multi-step workflows (RAG is just one application).
Abstraction Level	Higher-level, data-centric. Abstracts indexing, retrieval, summarization, query	Lower-level, workflow-centric. Breaks pipeline into many modular objects (Retriever, VectorStore, Memory, Chain, Agent).

	engines into one object (Index + Query Engine).	
Index Layer	Multiple index types: • VectorStoreIndex (semantic search over embeddings) • SummaryIndex (pre-summarized docs) • KeywordTableIndex (fast keyword lookup) • TreeIndex (hierarchical zoom-in).	No built-in indices. Relies on vector stores (Pinecone, FAISS, Chroma, etc.) or external DBs. Indexing is more manual.
Nodes	Explicit Node abstraction = smallest unit of knowledge (chunk of text + metadata). All indexes operate on Nodes.	No explicit "Node" object. Works with Document objects (text + metadata), but less central to the design.
Embeddings	Plug-and-play with any embedding model: OpenAI, Gemini, HuggingFace, Cohere, local models. Tightly coupled with Node & Index.	Also supports all major embedding models, but embedding step is usually explicit and user-controlled.
LLM Integration	Unified LLMPredictor interface. Works with OpenAI, Gemini, Anthropic, LLaMA, Mistral, etc. Often tied directly into query engines.	LLM and ChatModel wrappers. More mature ecosystem for model orchestration, supports async, streaming, structured outputs, function-calling.
Query Layer	Built-in Query Engines: • RetrieverQueryEngine • RouterQueryEngine • SubQuestionQueryEngine All extend how queries are broken down, routed, and answered.	Retrieval handled by Retrievers, then combined in Chains (like RetrievalQAChain) or Agents. More explicit chaining of steps.

Post-Processing (Rerank/Dedup/Compress)	First-class citizens: re-ranking, deduplication, compression, context budgeting are integrated into query pipeline.	Available but modular: handled via retriever wrappers (ContextualCompressionRetriever, reranker add-ons, custom filters).
Prompt Templating	Templates designed around retrieval (Base Prompt + Refine Prompt + Tree Summarization Prompt). Optimized for RAG and indexing tasks.	Flexible PromptTemplate, ChatPromptTemplate, OutputParser. More general-purpose, not RAG-specific.
Memory	Differentiates Chat History (raw transcript) vs Memory Store (facts distilled into Nodes). Retrieval can pull from both.	Multiple memory classes: ConversationBufferMemory, SummaryMemory, KGMemory. Designed more for conversation than structured retrieval.
Agents / Tool Use	Limited agent support. Focuses instead on Query Routing and Sub-Questioning.	Very strong agent ecosystem. Tools, multi-step reasoning, decision-making pipelines (e.g., ReAct, MRKL, AutoGPT-style).
Best At	• RAG pipelines • Multi-index querying (summary + vector + keyword + tree) • Document ingestion, chunking, and knowledge management.	• Complex workflows • Tool use & agents • Connecting LLMs to APIs, SQL, search engines, calculators, etc.
Ease of Use	Higher-level, fewer objects to wire together. You define an Index + Query Engine, and it "just works."	More flexible but verbose. You wire together retriever → chain → memory → prompt → LLM → output.
Learning Curve	Easier to start if focus is only RAG / knowledge QA.	Steeper if starting with RAG, but more powerful if building agents or multi-tool workflows.

When to Use Alone	When you want fast, production-ready RAG with good defaults for indexing & querying.	When you need agent workflows, multi-tool orchestration, or fine-grained control over every step.
When to Use Together	Very common: use LlamaIndex for ingestion + retrieval, wrap its Query Engine as a Tool inside LangChain Agent. Best of both worlds.	

2. When to use both (and why)

Use **both** when your app needs **organized, high-quality retrieval over your own documents** *and* **smart orchestration across multiple tools/steps**. In other words:

- You have real data (PDFs, HTML, wikis) and need **solid RAG**: clean chunking, metadata, summaries, reranking, dedup, context budgeting → **LlamaIndex** shines here.
- You also need the assistant to **do things beyond retrieval**: call external APIs, search the web, run a calculator, hit a SQL database, or follow a multi-step plan → **LangChain** shines here.

Think: **LlamaIndex = organized library**. **LangChain = assistant that decides when to use that library vs something else.**

2.1 Who does what (division of responsibilities)

LlamaIndex handles (data & retrieval quality)

- **Ingestion & structuring**: Load PDFs/HTML/TXT, split into **Nodes** (smart chunks), attach metadata (title, page, section).
- **Indexing**: Build the right **index types** (VectorStoreIndex for semantic, SummaryIndex for overviews, TreeIndex for hierarchical drill-down, KeywordTableIndex for exact terms).
- **Embeddings**: Choose and manage the **embedding model** used at index & query time (must match).
- **Query engines**: Provide **RetrieverQueryEngine / RouterQueryEngine / SubQuestionQueryEngine** to shape retrieval.

- **Retrieval upgrades**: **Reranking, deduplication, compression, context budgeting**, optional multi-query/HyDE/self-query routing.
- **Prompting for RAG**: Grounded Text-QA + Refine prompts, citation style, output shape for answers that actually cite sources.

LangChain handles (workflow & tool use)

- **Agents & tools**: Let the LLM decide **which tool to use next** (web search, SQL, code-runner, calendar API, etc.).
- **Multi-step plans**: "Search → fetch docs → compute → call API → summarize → answer."
- **Routing & policies**: If query is about live data (prices/news) → use web tool; if it's about your docs → call the LlamaIndex query engine.
- **App-level glue**: Streaming to UI, retries, tracing, eval with LangSmith, caching, rate-limit handling, secrets/config orchestration.

2.2 Minimal Architecture (Mental Sketch)

- **User question → LangChain Agent**
  - Reads the question and decides: "Is this about our documents or an external task?
- If **docs question**:
  - Agent calls a **Tool** that wraps **LlamaIndex Query Engine** (e.g., `ask_corpus(query)`).
  - LlamaIndex handles retrieval (rerank, dedup, compression, budgeting) and returns a grounded, cited answer + source nodes.
- If **external task**:
  - Agent calls other tools (web search, SQL, calculator), possibly combines with a LlamaIndex call if it needs both.
- **Final answer**:
  - Agent stitches results, enforces format (JSON/text), and streams to UI.

Result: **LangChain decides the route; LlamaIndex guarantees retrieval quality**.

2.3 Typical use cases where both are better together

- **Enterprise assistant**:
  Needs to answer from internal docs (policies, SOPs) **and** book meetings, query Jira, search the web for latest news.
  - LlamaIndex → robust document answers with citations.
  - LangChain → tool calls (Calendar, Jira, Web), plan multi-step workflows.

- **Analytics copilot**:
  Ask questions about PDFs and **also** run calculations on a warehouse or a CSV.
  - LlamaIndex → retrieve relevant document guidance/specs.
  - LangChain → SQL tool/csv tool to compute numbers; combines both.
- **Research bot**:
  Pull from your curated corpus **and** fetch fresh web evidence.
  - LlamaIndex → "what does our internal research say?"
  - LangChain → "augment with recent public sources," dedupe, then synthesize.

2.4 Practical "who does what" checklist

Give to LlamaIndex

- Breaking documents into Nodes; attaching metadata.
- Building Vector/Summary/Tree/Keyword indexes.
- Choosing embeddings; ensuring same model for index + query.
- Reranking, deduplication, compression, context budgeting.
- Grounded QA prompts with strict citation rules.
- Sub-question decomposition *over your indexes*.

Give to LangChain

- Agent planning ("which tool next?").
- Non-document tools: web search, SQL, calculator, code exec, emails, calendars.
- Policy decisions: live vs static answer, cost thresholds, fallbacks & retries.
- Telemetry & eval: LangSmith traces, dataset evaluations.
- UI streaming and application lifecycle (auth, secrets, configs).

2.5 Integration pattern (simple & reliable)

- **Wrap LlamaIndex as a LangChain Tool**:
  - `Tool(name="ask_docs", func=lambda q: llamaindex_qe.query(q))`
  - The agent can call `"ask_docs"` whenever it detects a doc question.
- **Return both**: the **final text** and the **source nodes** (for citations).
  - The agent can format: "Answer… (Sources: S1, S3)" and your UI can link to titles/URLs.
- **Guardrails**:

- If the LlamaIndex answer includes “I don’t know from the provided documents,” the agent can **failover** to web search or ask a follow-up.

2.6 Common pitfalls & how to avoid them

- **Mismatched embeddings** (indexed with one model, queried with another)
  → Always set the same embed model for ingestion and query in LlamaIndex.
- **Over-long contexts** (token overflow, high cost)
  → Turn on compression + budgeting in LlamaIndex query engine before sending to the LLM.
- **Duplicate chunks from the same file**
  → Use rerank + dedup post-processors in LlamaIndex.
- **Agents over-calling tools** (cost/latency spikes)
  → In LangChain, set tool-use policies (budget caps, step limits, routing heuristics).
- **Uncited answers**
  → Enforce citation template in LlamaIndex’s Text-QA/Refine prompts and surface `source_nodes` in UI.

RAG Implementations with LlamaIndex and Langchain

1. Memory RAG

1.1 Mental model: three clean layers

a. Embedding / LLM layer (models & policies)

- **Embedding model (query-time):** `text-embedding-004` (768-d, Gemini).
  - Purpose: turn the user query into a vector *in the same space as the Pinecone corpus*.
  - Implication: every semantic search needs **one embedding call** per query. That’s unavoidable with this corpus unless you re-embed.
- **LLM (answer-time):** `gemini-1.5-flash`.
  - Purpose: synthesize a short answer **from retrieved context only**.
  - We set **temperature low** (0.2) for factual tasks.
  - We keep **response mode = "compact"** to guarantee **exactly one LLM generation per query** (no refine/tree).

- **Grounding policy (in prompt):**
  - "**Use ONLY the CONTEXT**; if unsupported, answer exactly: *'No support in retrieved context.'*"
  - Inline citations `[...]` like `[S1]` after each claim.
  - 4–6 sentences, no outside knowledge.
  - This **cuts hallucinations** because the model is explicitly told how to behave **and** what to say when evidence is missing.

b. Data / Index layer (where content lives)

- **Vector DB: Pinecone** index: `"coffee-hybrid"`.
  - **Vectors:** 768-d (embedded using `text-embedding-004`).
  - **Text key:** `"text"` (your raw passage/snippet).
  - **Metadata:** at minimum `heading` (title/section), optionally `url/source/file_path/doc_id`.
  - Why Pinecone only: keeps infra simple and fast; avoids local ingestion at query time.
- **Schema guarantees (what the retriever expects):**
  - Every record has:
    - `id` → unique,
    - `values` → 768-d vector,
    - `metadata.text` → the chunk,
    - `metadata.heading` (and optionally `url`/`source`/…).
  - Namespaces (optional) can segment corpora for tenants or environments.

c. Query layer (how questions are answered)

- **Retriever:** `VectorStoreIndex` wrapper over Pinecone; `similarity_top_k` ~ 3–8 (we used 5).
  - Retrieval is **non-LLM**; it costs one embedding call per query to produce the query vector.
- **Synthesizer:** single-shot (`response_mode="compact"`) → **one LLM call**.
  - We inject a **strict template**: grounding, citations, fallback string, and length bounds.
  - We also build a **sources map** (`[S1] → heading + URL`) for UI linking.

That's the core. Everything else (multiquery, self-query, routing, re-ranking, compression) are **pluggable query-time modules** that sit between **retrieval** and **synthesis** (or modify retrieval) without changing the data layer.

1.2 Exact dataflow (what happens on a query)

```
User question
    |
    ├─(A) Embedding call (004) → query vector
    |
    ├─(B) Pinecone similarity search → top-k matches (nodes)
    |       • 0 LLM calls
    |
    ├─(C) (Optional post-processing)
    |       • Rerank (local cross-encoder) → reorder
    |       • Deduplicate → remove near-identical chunks
    |       • Compress (LLM) → trim to query-focused spans
    |       • Budgeting → pack to a safe prompt size
    |
    ├─(D) Build CONTEXT with numbered tags [S1], [S2], ...
    |       + build sources_map for UI
    |
    └─(E) ONE LLM call (compact synthesis)
            • Uses prompt: "Use ONLY the CONTEXT; else output fallback"
            • Answer has inline [S#] citations
```

1.3 Memory RAG integration (LangChain + LlamaIndex)

We wrapped the LlamaIndex pipeline into a **LangChain Tool**:

- **Tool name:** `ask_docs`
- **func:** calls the LlamaIndex **retriever → grounded synthesizer**, returns:
  - **final text with inline citations**, and
  - **`<SOURCES_JSON>{...}</SOURCES_JSON>`** (machine-readable mapping of `[S#] → heading/url` for your UI).
- **Agent:** a ReAct-style LangChain agent (`ChatGoogleGenerativeAI`) with **ConversationBufferMemory**.
  - The agent decides **when** to call `ask_docs`.

- **Guardrail:** if tool returns fallback ("No support in retrieved context."), the agent asks a follow-up or could fail over to web search (when you add that tool).

Why this pattern is reliable

- Retrieval and synthesis are encapsulated in a **single tool**; the agent cannot "forget" to ground.
- The tool's output embeds both **human-readable** and **UI-friendly** citations.
- Memory sits **outside** the RAG context; it shapes follow-ups, not the evidence.

1.4 Code

```
# --- INSTALL (as neeed) ---
# pip install -U llama-index-core llama-index-vector-stores-pinecone
# pip install -U llama-index-llms-google-genai
llama-index-embeddings-google-genai
# pip install -U google-genai pinecone-client
# pip install -U langchain langchain-google-genai

# ========================== IMPORTS (all in one place)
==========================
import os, json, textwrap
from typing import Tuple, Dict

# ---- LlamaIndex (for RAG over Pinecone) ----
from pinecone import Pinecone
from llama_index.core import Settings, VectorStoreIndex
from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding

# ---- LangChain (agent + memory + tool wrapper) ----
try:
    from langchain.tools import Tool
    from langchain.agents import create_react_agent, AgentExecutor
    from langchain.prompts import ChatPromptTemplate,
MessagesPlaceholder
    from langchain.memory import ConversationBufferMemory
except Exception:
    # Back-compat paths
```

```
from langchain_google_genai import ChatGoogleGenerativeAI


#
============================================================================
===========
# 1) EMBEDDING / LLM LAYER
#    (match Pinecone: you embedded with text-embedding-004 → 768-d)
#
============================================================================
===========
Settings.embed_model = GoogleGenAIEmbedding(
    model_name="models/text-embedding-004",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
)
# LlamaIndex LLM (used only for grounded synthesis in the tool)
Settings.llm = GoogleGenAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
    temperature=0.2,
    max_tokens=600,
)

# LangChain chat LLM for the AGENT (same model for consistency)
agent_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    api_key=os.environ.get("GOOGLE_API_KEY") or
os.environ["GEMINI_API_KEY"],
    temperature=0.2,
)


#
============================================================================
===========
# 2) DATA / INDEX LAYER  (Pinecone ONLY; no local PDFs)
```

```
#
============================================================================
===========
INDEX_NAME = "coffee-hybrid"
TEXT_KEY   = "text"  # change if your field is e.g. "page_content"

pc       = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc_index = pc.Index(INDEX_NAME)

vstore = PineconeVectorStore(pinecone_index=pc_index,
text_key=TEXT_KEY)
li_index = VectorStoreIndex.from_vector_store(vstore)  # uses
Settings.embed_model



#
============================================================================
===========
# 3) QUERY LAYER -- LlamaIndex → wrapped as a LangChain Tool
("ask_docs")
#    - Retrieval first (no LLM)
#    - Single-shot, grounded prompt: "Use ONLY the CONTEXT"
#    - Inline citations like [S1]; we also return a machine-readable
sources map
#    - Guardrail: if unsupported → return the fallback string (agent
can failover)
#
============================================================================
===========

FALLBACK_STR = "No support in retrieved context."

def _li_grounded_answer(question: str, k: int = 5, per_source_chars:
int = 900) -> Tuple[str, Dict]:
    """Run LlamaIndex RAG over Pinecone and produce (answer_text,
sources_map)."""
    # 1) Retrieve (cost: ONE embedding call with 004)
    hits =
```

```
li_index.as_retriever(similarity_top_k=k).retrieve(question)

    # 2) Build CONTEXT with numbered tags + store sources mapping for
the UI
    parts, sources_map = [], {}
    for i, h in enumerate(hits, start=1):
        tag = f"S{i}"
        md = h.node.metadata or {}
        heading = md.get("heading") or md.get("title") or "Untitled"
        src = md.get("url") or md.get("source") or
md.get("file_path") or md.get("doc_id") or "N/A"
        snippet = h.node.get_content()[:per_source_chars]
        parts.append(f"[{tag}] {heading}\n{snippet}")
        sources_map[tag] = {"heading": heading, "source": src}
    context = "\n\n---\n\n".join(parts) if parts else "(no results)"

    # 3) Strict, short, procedural prompt (single LLM call)
    prompt = f"""
You are a concise specialty-coffee expert.

RULES:
- Use ONLY the CONTEXT below.
- If the answer is not fully supported by the CONTEXT, reply exactly:
"{FALLBACK_STR}"
- 4-6 sentences max.
- After each factual claim, add an inline citation like [S1], [S2]
matching the CONTEXT tags.
- Do NOT use outside knowledge.

QUESTION:
{question}

CONTEXT (numbered sources):
{context}

ANSWER (with inline [S#] citations):
""".strip()
```

```
    answer = Settings.llm.complete(prompt).text.strip()
    return answer, sources_map


# ---- LangChain Tool wrapper ----
def ask_docs_tool(question: str) -> str:
    """
    Minimal wrapper that returns a **string** for the agent to read,
but embeds
    machine-readable citations for your UI between <SOURCES_JSON>
tags.
    """
    answer, sources = _li_grounded_answer(question)
    src_json = json.dumps(sources, ensure_ascii=False)
    # If unsupported, agent will see the fallback text and can decide
to web-search / ask follow-up.
    return f"{answer}\n\n(Sources inline as [S#]. Full map
below)\n<SOURCES_JSON>{src_json}</SOURCES_JSON>"

ask_docs = Tool(
    name="ask_docs",
    func=ask_docs_tool,
    description=(
        "Answer questions STRICTLY from the Pinecone 'coffee-hybrid'
docs. "
        "Outputs inline citations like [S1], and includes
<SOURCES_JSON> with titles/URLs. "
        f"If you see \"{FALLBACK_STR}\", ask a follow-up question or
propose web search."
    ),
)


#
==========================================================================
===========
# 4) LANGCHAIN AGENT with MEMORY (Memory RAG)
#    - The agent chooses when to call the 'ask_docs' tool.
```

```
#    - If tool response == fallback, it asks a follow-up (guardrail).
#
================================================================================
===========
prompt = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You are a helpful assistant. Use the 'ask_docs' tool for
any question "
         "that might be answered from the user's coffee knowledge
base. "
         f"If the tool returns '{FALLBACK_STR}', ask a brief
follow-up question to clarify."),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
        MessagesPlaceholder("agent_scratchpad"),
    ]
)

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True,
)

agent = create_react_agent(agent_llm, tools=[ask_docs],
prompt=prompt)
executor = AgentExecutor(agent=agent, tools=[ask_docs],
memory=memory, verbose=False)

# ============================== EXAMPLES
================================
# 1) A doc-grounded question → agent calls ask_docs, returns
grounded answer + citations
resp1 = executor.invoke({"input": "What is turmeric coffee? Keep it
short."})
print("\n=== AGENT #1 ===\n", textwrap.fill(resp1["output"], 100))

# 2) A likely unsupported question → tool returns fallback → agent
```

```
asks follow-up
resp2 = executor.invoke({"input": "Tell me about Ethiopian tea
ceremonies from 1800s."})
print("\n=== AGENT #2 ===\n", textwrap.fill(resp2["output"], 100))

# 3) Another doc question leveraging conversation memory (Memory RAG
flavor)
resp3 = executor.invoke({"input": "And ideal brewing temperature
guidance?"})
print("\n=== AGENT #3 ===\n", textwrap.fill(resp3["output"], 100))

# NOTE:
# - Each tool use = 1 LLM generation (synthesis) + 1 embed call for
retrieval.
# - The inline <SOURCES_JSON> lets your UI map [S1],[S2],... to
headings/URLs.
# - If you later add a web-search tool, the same agent can fail over
automatically
#   when it sees the fallback string.
```

Conclusion

- Today we zoomed out from the nuts and bolts of RAG and saw **why LlamaIndex exists at all**. Last time, we built pipelines manually and with LangChain — great for wiring workflows. But we also noticed the blind spot: a **vector DB alone is not always enough**.
- LlamaIndex fills that gap by acting like a **knowledge manager**. It doesn't just store text; it organizes it into **Nodes**, builds different **indexes** (semantic, summary, keyword, tree), and lets you query them through a flexible **Query Layer**. It ensures your LLM sees the right lens of the data — whether that's a precise keyword hit, a high-level summary, or a semantic match.
    - The big picture is this:
    - **LangChain** is the workflow brain that decides *how steps connect*.
    - **LlamaIndex** is the librarian that decides *how knowledge is best stored and retrieved*.
- Together, they give you both **precision** (structured retrieval) and **power** (multi-tool orchestration).

- So, as you go forward, think of LlamaIndex not as “just another RAG tool,” but as the layer that **elevates retrieval beyond vectors**. It’s what turns a simple search box into a genuine **knowledge assistant**.

codingninjas